Framework for Evaluation and Comparison of Integer Factorization Algorithms

Cristina-Loredana Duta, Laura Gheorghe, Nicolae Tapus
Department of Computer Science and Engineering
University Politehnica of Bucharest
Bucharest, Romania
cristina.duta.mapn@outlook.com, laura.gheorghe@cs.pub.ro, nicolae.tapus@cs.pub.ro

***Abstract*—Mathematicians have always been fascinated by the subject of finding the prime numbers of large composite numbers. They have studied various methods and have focused on developing several theories. Nowadays, this problem is of practical importance because the security of the public key cryptosystems depends on the difficulty of factoring public keys. The integer factorization algorithms have rapidly extended and improved such that today it is easy to factor a 100-decimal digit number and feasible to factor 155-decimal digit number. But the problem of factoring large integers still remains difficult because none of the algorithms run in polynomial time and numbers with more than 150 decimal digits are hard to factor. In this paper, we describe the implementation and performance of several integer factorization algorithms, in order to determine which is more efficient: trial division, Euler's, Fermat's, Pollard's p-1, William's p+1, Pollard Rho, Elliptic Curve Method, Shank's square forms, Dixon's, Continued Fraction Factorization, Quadratic Sieve, Number Field Sieve, Special Number Field Sieve and General Number Field Sieve. We implemented the algorithms and calculated their approximated computational runtime for input numbers of various sizes. Then, we mathematically analyzed their time complexities and then, to determine their performance, we compared the results of each algorithm with each other. For the implementation of the algorithms we have built an evaluation framework that contains the previously mentioned factorization algorithms and that allows the user to load data of different input size.**

Keywords—public key cryptography; prime numbers; composite numbers; integer factorization algorithms

I. INTRODUCTION

Since ancient times, an important role in the development of mathematics has been played by prime numbers. The concept of factoring numbers into prime numbers appeared in 300BC, when Euler demonstrated that there are infinitely many primes and defined the Fundamental Theorem of Arithmetic [1], according to which every integer can be defined as a unique product of prime factors. At the beginning, the factoring methods where used in mathematics. In 1640, Fermat was the first to define some concepts regarding factoring integers, ideas that can be seen in today's fastest methods. Euler continued the study, and in 1750, added some new ideas (he tried to understand integers of special forms). But the breakthrough for modern factorization algorithms was done by Legendre in 1798, when he presented his theory of congruent squares. In the same time, Gauss was developing a method, similar with the sieve of Eratosthenes. Gauss was the first to clearly state and prove, taking into consideration Euclid's theory, that any number has a unique prime power decomposition.

For a long time, factoring methods were seen as a part of pure mathematics, without any practical applications. Later on, due to the introduction of public-key cryptography in 1970s and its wide usage, the study of integer factoring methods and the design of faster factoring algorithms became an essential subject in cryptography. In 1970s it has been demonstrated that if the factoring problem is efficiently solved, the Rivest-Shamir-Adelman (RSA) cryptosystem would become insecure.

Until 1970, using the computer built by Morrison and Brillhart, the researchers were able to factor a composite number of 39 digits. Then, it 1980s, factoring became important work and the continued fraction method developed by Morrison-Brillhart, led to the factoring of a 70 digit composite number. In 1990s, with the appearance of the quadratic sieve algorithm, the largest number factorized was of 116 digits. This led to the RSA-challenge in 1994, when a 129-digit RSA number was factored using parallelization (more than 1600 computers were used). The quadratic sieve was replaced by the number field sieve method, which established a new record: factorization of a 200 decimal digits RSA number. After this, a 248 decimal digit integer was factored using General Number Field Sieve (GNFS). The last factored number was published in 2009 and it is RSA-768 which has 232 decimal digits.

The purpose of this paper is to provide a basic understanding of several types of integer factorization methods and to correctly establish which algorithm is best suited for real world applications. Taking into consideration this fact, we have implemented and optimized 14 factorization algorithms and based on the experimental results, we presented their performance for input numbers of various sizes.

This paper is organized as follows. In Section 2 we present the necessary background for our work. Section 3 includes the description of related work. An overview of the architecture for our evaluation framework is presented in Section 3. Section 4 includes details about our implementations of the framework and of the factorization algorithms. The metrics taken into consideration for the algorithms are specified in Section 5 together with the experimental results obtained. Our conclusions and future work are shown in Section 6.

978-1-4673-8460-5/16/$31.00 ©2016 IEEE

II. BACKGROUND

One of the oldest branches of mathematics is number theory. Integer factorization is a part of number theory which takes a composite number and determines its prime factors. In this section, some of the concepts from number theory are described and details of the integer factorization methods which we compared are offered, in order to have a better understanding of them.

According to mathematical theories, there are two categories of factoring methods: general-purpose algorithms and special-purpose algorithms.

Special-purpose algorithms – their runtime depends on properties of the number being factored or on its unknown factors, such as size, special form, etc. Factoring methods such as Pollard's p-1, William's p+1, Fermat's, Euler's are special-purpose algorithms. A subclass of this category includes those algorithms whose running time is dependent on the size of the smallest prime factor. For example, trial division is such an algorithm.

General-purpose algorithmswhose speed does not depend on the size of the prime factors, the number of prime factors or on the form of the number. The algorithms from this category are used to factor RSA numbers. The basic method on which most of the general-purpose factoring methods rely is the "congruence of squares". This category includes algorithms such as: Dixon's, Continued fraction factorization, Quadratic sieve, General number field sieve, Shank's square forms and so on.

A **perfect number** is defined as a positive integer which is equal to the sum of all its positive divisors other than itself. A **composite number** is defined as any positive integer, greater than one, which is not a prime number. A **Mersenne number** has the form $2^p - 1$, where p is a prime number. A Fermat **number** has the form $2^m + 1$ and if this number is a prime, than m must be of the form $m = 2^n$, so the final form of a Fermat number is $2^{2^n} + 1$.

Trial divisionis the simplest method of factoring [2]. In this method, the first step is to divide the given input number N, with all primes up $\sqrt{N}$beginning with 2 and collecting all divisors. It is a very simple algorithm, but it takes more than $O(\sqrt{N})$ bit operations, which is extremely inefficient for large composite integers (because it increases exponentially with the size of the input number).

Fermat's factorization algorithm [3] was discovered in the 17th century and is based on the idea that a composite number N can be rewritten as a difference of squares: $N = x^2 - y^2$. Using this difference, we can immediately obtain the factorization of N,$N = (x + y)(x - y)$. So Fermat's idea is to try various pairs of (x, y) until the first equality is verified. Fermat's method tries all integers from $\sqrt{N}$to N, which means that it is a very slow algorithm and in the worst case scenario it takes $O(\sqrt{N})$ steps to determine the factors, being in this case, even worse than trial division. On the other hand, if Fermat's method is applied to the modulus N of RSA [4] (the two prime factors of it are close to each other), the algorithm works fast.

Euler's factorization method [5] is based on the idea that a composite number N can be expressed as a quadratic form in two different ways. Compared with Fermat's method, Euler's algorithm is more effective for numbers which have factors not close to each other and also is much more efficient in comparison with trial division if the representations of numbers as sums of two squares can be made easily. Euler's factorization algorithm has unfortunately a great disadvantage: it doesn't work for integers which have any of the prime factors of the form $4k + 3$ raised to an odd power, because such a number cannot be written as a sum of two squares.

Pollards p-1algorithm was developed in 1974 by Pollard [6] and is a special-purpose factoring method which is based on Fermat's Little Theorem [7]. This method is best suited for integers that have a factor p where p-1 is B-smooth for a relatively small B. If this algorithm is used on non B-smooth integers it will lead to failure. The runtime of this method is $O(B\frac{\ln N}{\ln B})$. The algorithm is very efficient when p-1 has all small prime numbers. The worst case scenario is the following: $(p - 1)/2$ is prime, which means that Pollard's p-1 algorithm is no better than trial division.

Pollard's Rho is a special purpose factorization method, which was created in 1975 [8]. It factors a composite number N by iterating a polynomial modulo N. Even though it has some similarities with Pollard's p-1 factoring method, this algorithm contains a better way to choose the trial divisors and it's specialized on composite numbers with small factors. The runtime of the algorithm is $O(\sqrt{p})$, where p is N's smallest prime factor. This means that against an RSA modulus N with balanced primes the runtime of the algorithm is $O(N^{1/4})$, making it an inefficient method. This algorithm was refined in 1980s by Brent [9], which created a version approximately 25% faster than the original Pollard rho method.

William's p+1 algorithm is a variant of the Pollard's p-1 method, from the family of algebraic-group factorization algorithms, published in 1982 [10]. This method uses Lucas sequences [11] to perform multiplications over a quadratic field and to obtain a rapid factorization if one factor p of N can be decomposed in (p+1) small prime numbers.

Elliptic curve method (ECM) was designed by Lenstra in 1985 [13] and was refined shortly after by Brent and is the fastest of the special-purpose factorization algorithms. It's an improvement on Pollard p-1 algorithm which replaces the restriction of only using the group $\mathbb{Z}/p\mathbb{Z}$ with using the group of points on a random elliptic curve. Even though the operations of an elliptic curve group are more expensive, it offers the possibility to several groups at the same time. A careful analysis of the ECM algorithm shows that it has a time complexity of $L_p[1/2,1]$ (p is the smallest factor of N) based on heuristic estimates, which is much better than earlier methods.

Dixon's method was proposed in 1981's by Dixon [15] and represents a simplified version of the Morrison-Brillhart method [16]. It is based on Fermat's method (congruence of squares) and tries to find x and y such that $x^2 = y^2 \pmod N$. The factorization algorithm consists of two steps: data

978-1-4673-8460-5/16/$31.00 ©2016 IEEE

collection and data analysis. A detailed analysis shows that choosing a small factor base is much worse than too large, because even if we need only a few pairs, the chances of finding one pair are small. The optimal complexity of Dixon's factoring method is $O(\exp(2\sqrt{2}\sqrt{\log n \log\log n}))$, which is not as good as Elliptic curve method.

Continued fraction factorization (CFRAC) algorithm it's a general purpose factorization method, which was described first in 1931, but it was Morrison and Brillhart that developed it as a computer algorithm in 1975 [16]. This method brings a new idea of how to find the desired squares, which is using continued fractions. The convergents of the continued fraction of $\sqrt{N}$ are used in CRFAC method and are kept as relations those values that are smooth over the factor base. CFRAC method has a time complexity of $O(e^{\sqrt{2 \log n \log\log n}})$ and one of its disadvantages is that it performs a lot of useless divisions.

Quadratic sieve method was proposed in 1981 by Pomerance and is a general-purpose factorization method [17]. It's a combination of Fermat's method and Dixon's method and avoids a large part of the useless divisions which were done in CFRAC, becoming in this way, the second fastest method known. QS is based on the idea of congruent squares and uses quadratic polynomial and quadratic residues to find the squares. QS is still the fastest method for factoring numbers with less than 100 decimal digits. The time complexity of QS is the same as for ECM, but the ECM operations are more expensive, which is why QS is faster than ECM.

Number field sieve is a general purpose factorization algorithm which was published in 1988, by Pollard and was improved by Promerance, Lenstra [18] and many others. It is considered to be the fastest factoring method known today, but because of its overhead and increased complexity, it is faster than QS only for composite integer numbers which have more than 110-120 decimal digits. NFS has many similarities with QS and can be expressed as an expansion of the QS in which polynomials with a degree greater than 2 are used. There are two variants of NFS, the special number field sieve (SNFS) [19] and the general number field sieve (GNFS) [20]. The difference between SNFS and GNFS is that the first one works only on a special type of composites ($r^e \pm s$, where r and s are small) and the second one can be applied for all types of composites.

Shanks square forms factorization algorithm (SQUFOF) was developed by Shanks at the end of the 19th century and represents an improvement of the Fermat's factorization method [21]. This algorithm consists of finding pairs (x, y) which satisfy the relation $x^2 \equiv y^2 (\mathrm{mod} N)$. SQUFOF works for integers which are at most $2\sqrt{N}$ and can be efficiently implemented to factor composite integers of 62 bits (it is small enough to be implemented on programmable calculator). The time complexity for SQUFOF is $O(\sqrt[4]{N})$.

III. Related Work

To give a better perspective about the importance and the utility of the framework we have created for evaluating and comparing integer factorization methods, this section presents various methods and techniques for evaluating factoring methods and the results obtained from other solutions.

In 1994, Montgomery [22], published a paper describing the modern integer factorization algorithms such as trial division, Pollard Rho, Pollard p-1, Step 2, William p+1, Elliptic Curve method, CFRAC, Quadratic Sieve, Multiple Polynomial QS (MPQS), NFS. The author also presented the improvements in technology until 1994, the factorization of RSA-129 number and 162-digit Cunningham number.

Bimpikis and Jaiswal [23] present in their paper a survey regarding modern factoring algorithms such as Pollard's Rho, QS and NFS. Their description was related to the security of RSA cryptosystem and they showed that the best algorithm to attack RSA is NFS and the use of distributed computing.

In [24], the authors have chosen for evaluation three factorization methods: Dixon's algorithm, Fermat's factorization method and Pollard Rho's algorithm. Their experimental results showed that Pollard Rho's algorithm is faster than the other two methods. Dixon's algorithm was faster than Fermat's factorization method due to the complexity of the factor base, as they mentioned.

In [25] the author performs an experimental comparison of several factorization algorithms for composite numbers from 50 to maximum 200 bits. He implemented factorization algorithms such as SQUFOF, MeKee's speedup of Fermat's algorithm, CFRAC algorithm, QS and Self-Initializing QS (SIQS). According to his results, SQUFOF had the best performance for composite numbers up to 64 bits and for numbers with more than 64 bits, SIQS offered the best results.

There are some available factoring software that can be used to compare the performance of integer factorization algorithms. MIRACL [26] stands for Multiprecision Integer and Rational Arithmetic C/C++ Library. This software includes factoring methods such as Pollard's Rho, Pollard's p-1, William's p+1, elliptic curve, MPQS. Its latest version is able to factor all numbers up to 80 decimal digits. LiDIA [27] is a C++ API who has a factoring program that contains the following methods: trial division, elliptic curve method, MPQS.

CALC [28] is software written in ANSI C and Yacc which has many built-in functions for number theory, including factorization algorithms such as Pollard's p-1, elliptic curve method, MPQS. All factoring methods work only for integer numbers up to 55 decimal digits.

Schulenerg and Associates [29] have developed a software program that can perform: trial division and Pollard's Rho factoring method (for numbers with less than 200 digits) and MPQS (for integers of maximum 100 digits).

IV. Framework Architecture

The architecture and the design principles for the framework we have developed for evaluating and comparing integer factorization algorithms are described in this section. A use case diagram with 2 actions, which can be seen in Figure 1, was created to illustrate the interactions between the users and the framework.

978-1-4673-8460-5/16/$31.00 ©2016 IEEE

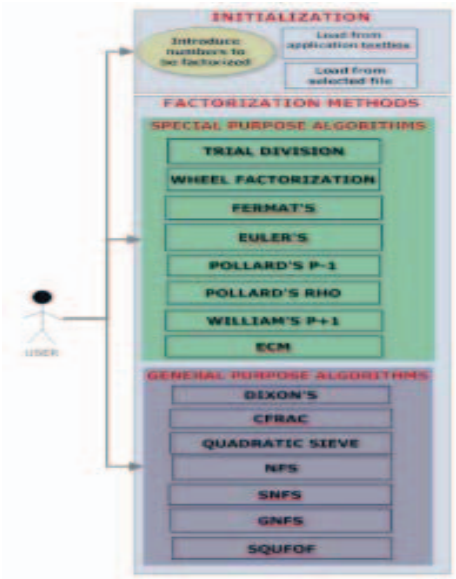

Fig. 1. Use case diagram

The first action in the use case is the initialization action, which offers two options: introduce in the corresponding field in the framework the composite number which is going to be factorized or load a file that contains several large integer numbers which the user wants to factorize. If the initialization step is correctly performed, the second step, applying integer factorization methods action, becomes available to the user. The design of the framework allows the user to select which factorization algorithm he wants to apply, giving him the possibility to take into account also the type of the algorithms (special-purpose or general-purpose factorization methods). The user can run all factorization algorithms, or he can select only those methods that are relevant to him. The results of the algorithms are written in text files, which are generated automatically by the framework, but are also shown directly in the graphical user interface.

As for implementation details, the framework was developed under Microsoft Windows Operating System with .NET 4 Framework Client Profile, using as a design the "thick client application" model. We chose this design taking into account several aspects such as: the framework had to be easily deployed on standalone computers with low deployment costs and moreover, the framework had to be easy to maintain by anyone. Our main was to allow anyone who is interested in integer factorization methods to use for analysis and for better understanding of the methods, our framework. If the user wants to use and run the framework on parallel and distributed systems, this can be achieved easily, due to the fact that we have designed the framework so that it requires minimum code changes in this situation.

The design pattern used for the framework's implementation was Model View Presenter (MVP), which has three layers, the Graphical User Interface (GUI), Application logic and Model business and it was the best option because it isolates the model layer from the view layer. Figure 2 illustrates the GUI of our framework.

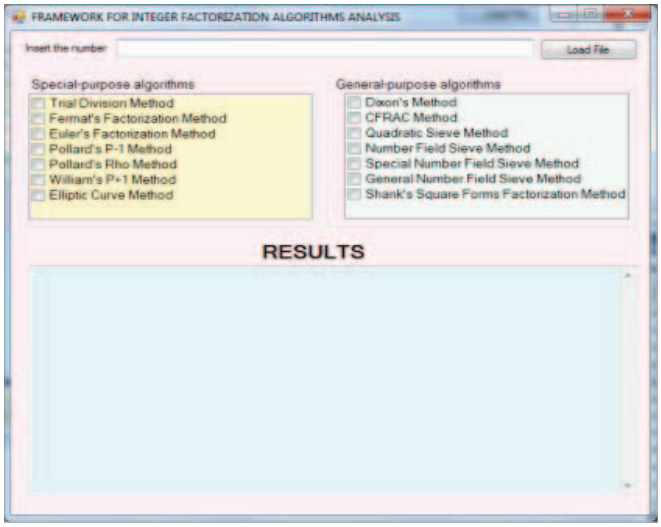

Fig. 2. GUI of the framework

As it can be seen, the factorization algorithms are classified into two groups, "Special-purpose algorithms" and "General-purpose algorithms", each group containing 7 methods. The framework handles all possible input errors and for each of them it displays corresponding messages. For instance, if the user introduces letters instead of digits in the textbox corresponding to the number that is going to be factored, a message will pop up "The input number is incorrect!"

Figure 3 shows the results displayed by the framework when trying to factor the number 17017 for the factorization algorithms selected by the user, which are: Fermat's, Euler's, ECM and SQUFOF methods.

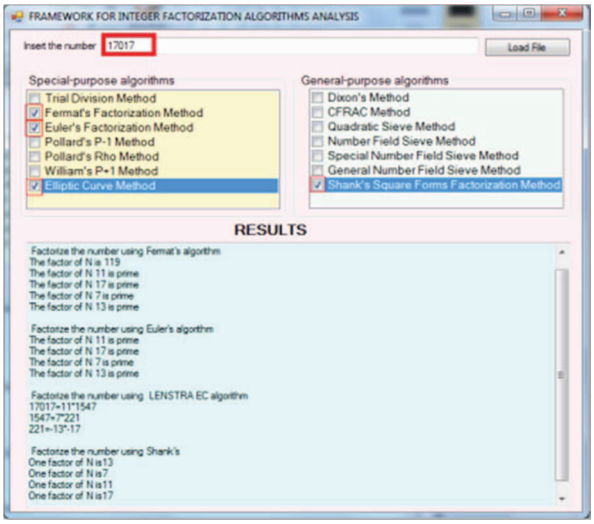

Fig. 3. Example of results for number 17017

V. Experimental Evaluation

The purpose of this paper is to determine which integer factorization algorithm is more efficient and is best suited for real world applications. In the evaluation process, we have used a system with Intel Core i5-5250U Processor, CPU 1.6 GHz, 16 GB of RAM, Windows 8 32-bits and video card integrated.

978-1-4673-8460-5/16/$31.00 ©2016 IEEE

As input for the framework, we used composite numbers of various sizes- starting from 2 and up to 50 digits. The output is represented by the results of the factorization methods (that display the prime factors for the given input number) and also by the elapsed time necessary for each algorithm to finish the prime decomposition. We have performed the comparison of the algorithms taking into consideration their types special-purpose and general-purpose factorization algorithms. The generic composite numbers given as input data were selected randomly and the special form composite numbers were taken from [30]. All these numbers can be seen in Table 1.

TABLE I. GENERIC AND SPECIAL FORM NUMBERS GIVEN AS INPUT

# of digits	Number	Generic / Special form
1	9	Fermat
2	33	Fermat
3	129	Fermat
4	2047	Mersenne
5	16385	Fermat
6	524287	Mersenne
7	2608837	Generic
8	12809161	Generic
9	128894237	Generic
10	4294967297	Fermat
11	11944043021	Generic
12	137438953471	Mersenne
13	1019829472003	Generic
14	65215596741409	Generic
15	106703288395397	Generic
16	7683116086849193	Generic
17	11821532550681719	Generic
18	576460752303423487	Mersenne
19	1807999095332691337	Generic
20	18446744073709551617	Fermat
25	5565990898925529810152251	Generic
30	280829369862134719390036617061	Generic
39	340282366920938463463374607431768211457	Fermat
40	2425967623052370772757633156976982469682	Generic
50	54987541231012456465487987654132654568741532457817	Generic

Table 2 contains the results for each special-purpose factorization algorithm and Table 3 contains the results of the general-purpose factorization algorithms. More exactly, the two tables contain the time (in seconds) necessary to factorize the given input number. We have only displayed the results for numbers with more than 12 digits for Table 2 and with more than 20 digits for Table 3, because integers of smaller length did not provide relevant results. If the symbol "∞" is seen in the time column, it signifies that it takes too long to produce the results (determine the factors of the given input number), more than 1 hour or that the algorithm failed during the factorization process. If the value "0" appears, this means that the algorithm realizes the factorization of the given composite number almost immediately.

TABLE II. GENERIC AND SPECIAL FORM NUMBERS GIVEN AS INPUT

# of digits	Trial Division	Fermat's	Euler's	Pollard's P-1	Pollard's Rho	William's P+1	ECM
13	2.118	0.104	0.0963	0.823	0	0.009	0
14	2.714	0.645	0.589	1.487	0	1.395	0
15	4.688	4.312	4.168	0	0	0	0.577
16	5.092	4.217	4.105	5.914	0	5.711	1.532
17	7.274	7.183	7.169	0	0	0	1.94
18	10.049	8.820	8.614	0.246	0	0.156	2.732
19	14.539	12.512	11.180	0.368	0	0.329	3.234
20	25.348	22.156	20.882	0.872	0.193	0.726	7.164
25	106.051	97.219	94.663	1.253	0.246	1.016	24.871
30	693.714	638.004	611.095	722.558	0.519	678.231	202.773
39	∞	2543.181	2289.371	1184.201	1.611	967.920	456.518
40	∞	2799.330	2461.903	1243.005	2.825	1020.115	498.629
50	∞	∞	∞	3609.619	6.645	3289.476	864.919

TABLE III. GENERIC AND SPECIAL FORM NUMBERS GIVEN AS INPUT

# of digits	Dixon's	CFRAC	QS	NFS	SNFS	GNFS	SQUFOF
20	7.210	0.019	0	0	0	0	0
25	14.228	0.096	0.045	0.010	0.010	0.008	0.041
30	19.384	0.589	0.066	0.042	0.038	0.019	0.053
39	49.205	1.332	0.089	0.068	0.062	0.031	0.078
40	50.663	1.486	0.126	0.092	0.083	0.045	0.085
50	111.292	3.345	1.534	0.588	0.512	0.267	1.022

The results for the special-purpose factorization algorithms are presented in Figure 4. For the trial division method, it can be observed that the running time increases exponentially with the length of the input number, and factoring a number of 50 decimal digits is computationally expensive and even infeasible in some situations. The results of Fermat's algorithm are similar with the ones obtained for trial division, but slightly better, with this method being able to factorize input numbers of 40 digits, which was impossible with the previous algorithm. By comparing Euler's method to Fermat's it can be seen that the first one is more effective and is also much more efficient than trial division.

From Table 2 and Figure 4 it can be observed that Pollard's Rho algorithm efficiently factorizes composite numbers of small length. For instance, in less than 1 second it can factorize numbers up to 30 decimal digits. If we look at Pollard's p-1 algorithm's results, it is visible that for some values, the running time fluctuates, it does not constantly increase exponentially with the size of the input number. The main reason for these results is that the algorithm has a pseudo-random nature (is probabilistic). William's P+1 results show

978-1-4673-8460-5/16/$31.00 ©2016 IEEE

that this factorization algorithm is better than Pollard's p-1 method (there is a small difference between their running times), but is less efficient (thousands of times slower) in comparison with Pollard's Rho algorithm.

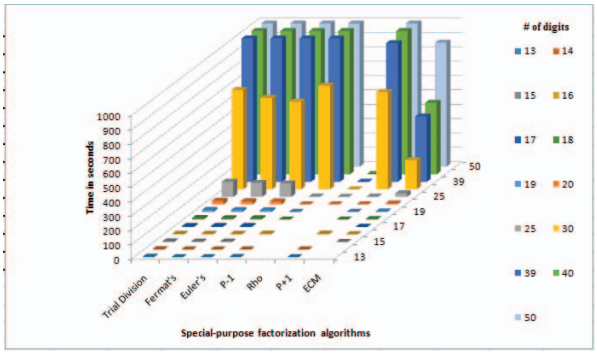

Fig. 4. Results of special-purpose factorization algorithms

Contrary to our expectations that ECM was the fastest factorization algorithm, it can be seen from the results that ECM is better than all methods, except Pollard's Rho algorithm. The explanation is the fact that it has been demonstrated by researches that ECM has low performance when it factorizes small length integers and that it has great performance for input number of more than 65 digits. This means that the performance of ECM is inverse proportional with the size of the input number (it increases when the size of the number grows).

From all these results it can be concluded that Pollard's Rho algorithm offers the best results, followed by ECM method. Also, it was shown that trial division is not a feasible solution when numbers with more than 30 have to be factorized.

In Figure 5 are presented the results for the general-purpose factorization algorithms, when the input numbers have more than 20 digits, because for numbers of smaller length the results aren't relevant as it can be seen in Table 3.

The slowest algorithm is Dixon's whose running time increases exponentially with the length of the input number. For instance, to factorize a number of 50 decimal digits the algorithm needs more than 100 seconds. CFRAC method is very efficient in comparison with Dixon's algorithm (is approximately a 100 times faster than Dixon's). The rest of the general-purpose factorization algorithms have great performance, all of them being able to factorize a 50 digit number in less than 1 second.

The differences between QS, NFS, SNFS, GNFS and SQUFOF are very small and can be better observed for numbers with more than 50 digits. Based on the results we obtained for the input numbers of various sizes, we can affirm the fastest general-purpose algorithm is GNFS, as it was expected.

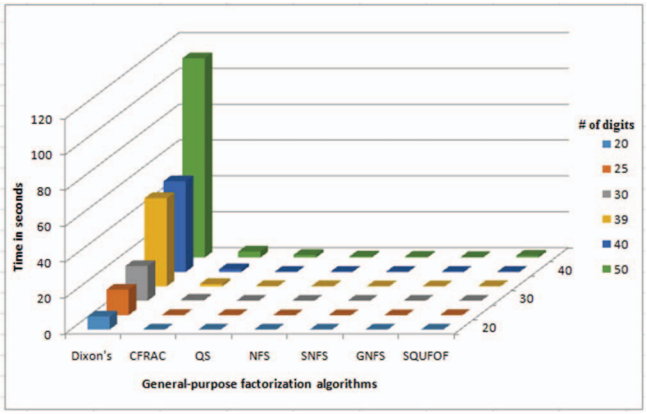

Fig. 5. Results of general-purpose factorization algorithms

VI. Conclusions

The purpose of this paper is to present a framework which can be used to evaluate and to compare integer factorization algorithms, based on input data given by the user. The framework allows the selection of files which contain numbers of different length, randomly chosen or with certain mathematical properties. In this context, we have implemented and analyzed 14 factorization methods (special-purpose and general-purpose algorithms). We have given as input, numbers with more than 20 digits and up to 50 digits. Certain characteristics can be observed from our results. For instance, Pollard's Rho method is the fastest special-purpose factorization algorithm and GNFS method has the best performance overall. It can also be seen the fact that general-purpose algorithm have better performance and are best suited for practical applications, such as factoring RSA numbers.Our future work will involve the optimization of all integer factorization algorithms, their deployment on parallel and distributed systems and the analysis and identification of practical scenarios where they are very useful.

Acknowledgment

The work has been funded by the Sectoral Operational Programme Human Resources Development 2007-2013 of the Ministry of European Funds through the Financial Agreement POSDRU/187/1.5/S/155536 and POSDRU/159/1.5/S/134398.

References

[1] M. D. Hendy, "Euclid and the fundamental theorem of arithmetic", in: Historia Mathematica, vol. 2, pp. 189-191, 1975.

[2] D. M. Bressoud, "Factorizations and primality testing", in: Springer-Verlag, pp. 21-22, New York , 1989.

[3] P. Lehman, R. Sherman, "Factoring Large Integers", in: Mathematics of Computation, vol. 28, pp. 637-646, 1974.

[4] J. McKee, "Speeding Fermat's factoring method", in: Mathematics of Computation, vol. 68, pp. 1729-1737, 1999.

[5] O. Oystein, "Euler's Factorization Method", in: Number Theory and Its History, pp. 59–64, 1976.

[6] J. M. Pollard, "Theorems of factorization and primality testing", in: Proceedings of the Cambridge Philosophical Society, vol. 76, pp. 521–528, 1974.

978-1-4673-8460-5/16/$31.00 ©2016 IEEE

[7] T. Nagell, “Fermat’s Theorem and Its Generalization by Euler”, in: Introduction to Number Theory, pp. 71-73, 1951.

[8] J. M. Pollard, “A Monte Carlo method for factorization”, in: BIT Numerical Mathematics, vol. 15, pp. 331–334, 1975.

[9] R.P. Brent, “An improved Monte Carlo factorization algorithm”, in: BIT Numerical Mathematics, vol. 20, pp. 176–184, 1980.

[10] H.C.Williams, “A p + 1 method of factoring”, in: Mathematics of Computation, vol. 39, pp. 225–234, 1982.

[11] L.E.Dickson, “Recurring Series; Lucas’ un, vn”, in: History of the Theory of Numbers, vol. 1, pp. 393-411, 2005.

[12] T. Nagell,“Jacobi's Symbol and the Generalization of the Reciprocity Law”, in: Introduction to Number Theory, pp. 145-149, 1951.

[13] Jr. H. W. Lenstra, “Elliptic curve factorization, personal communication via samuel wagstaff jr”, 1985.

[14] I. Blake, G. Seroussi, N. Smart, “Elliptic curves in cryptography”,Cambridge University Press,1999.

[15] J. D.Dixon, “Asymptotically fast factorization of integers”, in: Math. Comp., vol. 36, pp. 255–260, 1981.

[16] M. A. Morrison,J. Brillhart,“A method of Factoring and the factorization of F7”, in: Mathematics of Computation, vol. 29, pp. 183-205, 1975.

[17] C. Pomerance,“The quadratic sieve factoring algorithm”, in: Advances in cryptology, EUROCRYPT’84, vol. 209, pp. 169–182, 1985.

[18] A. K. Lenstra, H. W. Lenstra, “The Development of the Number Field Sieve”, in: Lecture Notes in Mathematics 1554, New York, 1993.

[19] R. D.Silverman, “Optimal Parameterization of SNFS”, in: Journal Mathematical Cryptology, vol. 1, pp. 105–124, 2007.

[20] C. Pomerance, “A Tale of Two Sieves” ,in: Notices of the AMS, vol. 43, pp. 1473–1485, 1996.

[21] D. Shanks, “Analysis and Improvement of the Continued Fraction Method of Factorization”, 2004.

[22] P. L. Montgomery, “A Survey of Modern Integer Factorization Algorithms”, in: CWI Quarterly, vol. 7, pp. 337-365, 1997.

[23] K. Bimpikis, R.Jaiswal, “Modern Factoring Algorithms”, a Technical Report Presented to the University of California, San Diego, pp. 1-15, 2005.

[24] D. Miller, V. Huber, M. Hagaman, C. Morrision, “Comparing the heuristic running times and time complexities of integer factorization algorithms”, in Supercomputing Challenge, Team 14, 2015.

[25] J. Milan, “Factoring Small Integers - An Experimental Comparison”, 2007.

[26] MIRACL library, https://www.certivox.com/miracl (Last Accessed: November 2015)

[27] LiDIA library, https://www.informatik.tu-darmstadt.de/de/fachbereich/ (Last Accessed: November 2015)

[28] CALC library, http://www.numbertheory.org/calc/krm_calc.html (Last Accessed: November 2015)

[29] Factorization software, http://www.schulenberg.com/ (Last Accessed: November 2015)

[30] N. J. A.Sloane, “Sequences A001348/N1079, A000051/N0266, A000215/N0990”, in: The On-line Encyclopedia of Integer Sequences

978-1-4673-8460-5/16/$31.00 ©2016 IEEE